Towards Retrieval-Augmented Large Language Model-Based Conversational Recommender System

Dayu Yang[1](✉) and Fuli Wang[2]

[1] University of Delaware, Newark, DE 19702, USA
dayu@udel.edu
[2] Georgia Institute of Technology, Atlanta, GA 30332, USA
fwang388@gatech.edu

Abstract. Conversational Recommender Systems (CRS) aim to provide recommendations through multi-turn natural language interactions. Recent LLM-based approaches have shown impressive performance by leveraging large language models (LLMs). However, these systems face several challenges: they struggle to recommend newly listed items, and are prone to hallucinations.

Retrieval-Augmented Generation (RAG) introduces up-to-date knowledge and mitigates hallucinations. Unfortunately, RAG cannot be directly implemented to CRS. First, we found retrieval quality is often hindered by the absence of high-quality query reformulation supervision. To resolve this, we present a pseudo query reformulation method called Compare-then-Summarize (CTS), which eliminates the need for costly manual annotations. CTS-generated queries are used to fine-tune or few-shot prompt an LLM, serving as the query reformulator during inference. Extensive experiments show that CTS supervision is a viable alternative to human annotations, significantly improving retrieval performance. While the retrieval stage reduces hallucination rates, we found some still persist. Moreover, integrating recommendation and response generation remains an open challenge. To tackle this, we introduce Rationalize-then-Recommend (RTR), a fine-tuning task that jointly generates both recommendations and their underlying rationale, ensuring only real items from the valid candidate space are suggested.

In Summary, we proposed retrieval-augmented LLM-based CRS that enable the implementation of RAG on CRS, and therefore advancing CRS from LLM-based to RAG-based. Comprehensive experiments confirm that our method consistently outperforms state-of-the-art LLM-based CRS across two widely-used datasets. We open-source our code at https://github.com/dayuyang1999/RAGCRS.

Keywords: Conversational Recommender System · Large Language Model · Information Retrieval System

© The Author(s), under exclusive license to Springer Nature Singapore Pte Ltd. 2025
X. Wu et al. (Eds.): PAKDD 2025, LNAI 15872, pp. 317–330, 2025.
https://doi.org/10.1007/978-981-96-8180-8_25

1 Introduction

Conversational recommender systems (CRS) capture user preferences through multi-turn natural language conversations, offering several advantages over traditional recommendation systems that rely on historical user actions [4]. CRS allows for explicit preference elicitation, providing immediate and accurate understanding [4], explanations for recommendations [8], and the ability to address complex needs via step-by-step interactions [8]. Recent advancements have highlighted the potential of large language models (LLMs) to enhance CRS through superior language understanding and natural language generation [6,19,23]. LLMs can accurately interpret subtle preferences from noisy conversations and generate fluent responses that elicit user preferences effectively [1,14,15].

However, directly prompting LLMs for recommendations introduces several limitations [23]. First, LLM-based CRS often generate items beyond the target dataset, complicating performance evaluation and risking recommendations from competitor platforms (e.g., recommending Amazon content on Netflix) [24]. Second, these systems struggle to recommend newly listed items, as they rely on static knowledge within model parameters [7]. Third, hallucinations persist, with studies reporting up to 5% of recommendations being fabricated, even with advanced models like GPT-4 [6]. Additionally, separating recommendation and response generation stages reduces system efficiency [6].

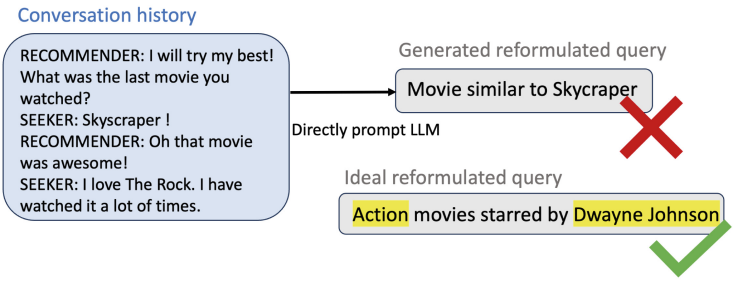

Fig. 1. Directly prompting LLMs generates queries lacking generalizability.

Retrieval-augmented generation (RAG) provides a promising way to inject fresh knowledge and mitigate hallucinations [3]. However, conversational histories tend to be long and noisy, while retrievers like DPR [10] require concise queries. Directly using raw conversation history for retrieval degrades performance [20]. A straightforward approach is query reformulation, where a model refines conversation history into effective queries [21]. However, zero-shot query generation by LLMs often yields semantically relevant but contextually inappropriate results. For example, as shown in Fig. 1, a zero-shot prompt for "Skyscraper" retrieves items like "tower" or "tall building" instead of the intended

movie starring Dwayne Johnson. Highlighted concepts with good generalizability are more effective for retrieval. Fine-tuning LLMs with supervision improves query reformulation, but collecting ground-truth labels is expensive due to the complexity of CRS data.

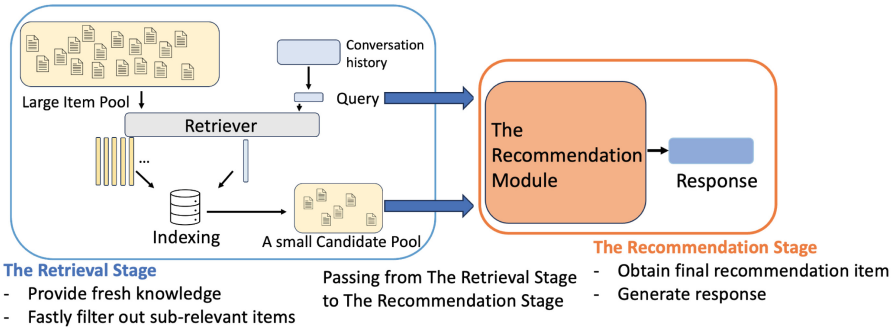

Fig. 2. The overall structure of RAGCRS.

While RAG reduces hallucinations, it does not eliminate them entirely [9]. Additionally, combining recommendation with response generation for both efficiency and coherence remains a challenge [27].

To address these challenges, we propose Retrieval-Augmented LLM-based Conversational Recommender Systems (RAGCRS), which improves RAG on CRS task. In the retrieval stage, we introduce a "Compare-then-Summarize" (CTS) approach to generate pseudo query annotations, serving as supervision for fine-tuning a query reformulator LLM. This enables effective retrieval, injecting fresh knowledge and reducing hallucinations. In the recommendation stage, we design a novel fine-tuning task called "Rationalize-then-Recommend" (RTR). RTR requires the LLM to provide rationales for recommendations and select the best item from a candidate list containing hard negatives and ground truth items. Our experiments show that RAGCRS consistently outperforms state-of-the-art LLM-based CRS while eliminating hallucinations.

2 Related Work

With the rapid advancement of dialogue systems, conversational recommender systems (CRS) have become a prominent research area. A major line of CRS research focuses on graph-based models [2,25,26], as CRS often target knowledge-intensive domains, such as movies and music, where essential knowledge is organized within knowledge graphs. To leverage these graphs, CRSs commonly use graph neural networks (GNNs) to learn node embeddings through supervised learning. However, graph-based CRSs struggle with language understanding. They extract user preferences by mapping conversation words to nodes

in the knowledge graph, relying on a "1-gram" approach that limits expressiveness. This dependency on pre-built graphs restricts CRS performance in accurately capturing nuanced preferences. For example, if a user says, "I do not like action movies before 2020", the CRS may still recommend action movies, ignoring the temporal constraint, as only the keyword "action" is represented in the graph, failing to capture the full semantic meaning.

Recently, LLMs [15] have been introduced into CRS for their strong language understanding and ability to model user preferences directly, reducing reliance on knowledge graphs [6]. However, existing LLM-based CRSs prompt the LLM with historical conversations to generate recommendations, which introduces three key challenges: (1) reliance on the LLM's internal knowledge limits recommendations for newly-listed items, (2) hallucinations may occur, generating non-existent or inaccurate content, and (3) it becomes difficult to control the candidate space, affecting recommendation quality [6].

3 Retrieval-Augmented LLM-Based CRS

In this section, we detail our proposed retrieval-augmented LLM-based CRS, starting with an overview, followed by specifics of the retrieval and recommendation stages.

3.1 Overview of the Approach

The overall architecture of RAGCRS is illustrated in Fig. 2, comprising two stages: retrieval and recommendation. The retrieval stage aims to efficiently filter out sub-relevant items, generating a small candidate pool. This pool introduces domain-specific, up-to-date knowledge, minimizing hallucination, and enabling recommendations for newly-listed items. Figure 3 presents an example interaction with a human movie seeker.

The retrieval stage utilizes a lightweight embedding model for efficient item matching via shallow interactions (e.g., dot product), which may sacrifice recommendation accuracy [12]. To address this, the recommendation stage refines the user's subtle preferences from noisy and lengthy conversations, delivering a final recommendation from the candidate pool. Leveraging LLMs, this stage combines item recommendation with response generation, allowing the model to select appropriate items and produce informative responses that guide users toward their desired choices [2,26].

3.2 The Improved Retrieval Stage: Compare-Then-Summarize (CTS)

The retrieval stage leverages a retriever to narrow the candidate set from millions to a few relevant documents. Using an offline encoder, each item description $\mathbf{d}_j \in \mathcal{D} = \{\mathbf{d}_1, \mathbf{d}_2, \ldots, \mathbf{d}_N\}$ is embedded as $\mathbf{e}_{d_j} = \text{Encoder}(\mathbf{d}_j)$, indexed via partitioning or tree-based methods for approximate nearest neighbor search [22].

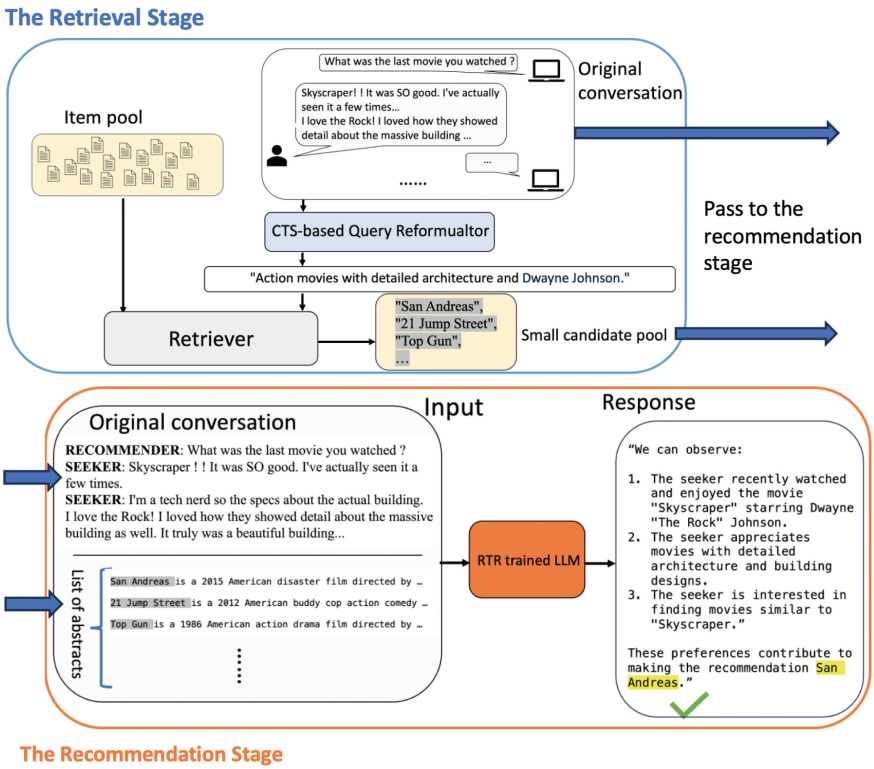

Fig. 3. An example implementing RAGCRS, highlighting the final recommendation in yellow. (Color figure online)

At turn i, the user message u_i and conversation history C_i generate a query q_i, embedded as $\mathbf{e}_{q_i} = \text{Encoder}(q_i)$. The cosine similarity $\text{sim}(\mathbf{e}_{q_i}, \mathbf{e}_{d_j}) = \frac{\mathbf{e}_{q_i}^\top \mathbf{e}_{d_j}}{\|\mathbf{e}_{q_i}\| \|\mathbf{e}_{d_j}\|}$ is computed between the query and each item embedding. The top K items with the highest similarity scores form the candidate set $\mathcal{D}_i^{\text{top}} = \arg\text{top}_K(\{\text{sim}(\mathbf{e}_{q_i}, \mathbf{e}_{d_j}) \mid \mathbf{d}_j \in \mathcal{D}\})$, forwarded to the recommendation stage (Fig. 2).

Retrieval accuracy heavily depends on the quality of the reformulated query q_i, especially with long or noisy conversations [20,21]. However, generating effective reformulations often requires costly human annotations.

We propose the **Compare-Then-Summarize (CTS)** method to generate supervision efficiently. For a training instance i, given ground truth items $\{p_1^i, p_2^i, \ldots\}$, the original conversation C_i, and their descriptions $\{\mathbf{d}_{p_1^i}, \mathbf{d}_{p_2^i}, \ldots\}$, a language model LLM_1 summarizes commonalities to produce a concise query $s_i = \text{LLM}_1(C_i, \{\mathbf{d}_{p_1^i}, \mathbf{d}_{p_2^i}, \ldots\})$.

These pseudo queries s_i supervise the training of a second model LLM_2, tasked with generating s_i based solely on C_i: $\hat{s}_i = \text{LLM}_2(C_i)$. We train LLM_2

by minimizing the negative log-likelihood over the token sequence:

$$\mathcal{L}_{\text{CTS}} = -\sum_{i}\sum_{t=1}^{T_i} \log P_{\text{LLM}_2}(s_i^{(t)} \mid s_i^{(<t)}, C_i),$$

where $s_i^{(t)}$ is the t-th token, T_i the sequence length, and $s_i^{(<t)}$ all preceding tokens.

During inference, LLM_2 generates the reformulated query $q_i = \text{LLM}_2(C_i)$, which is used for retrieval as described earlier.

By leveraging CTS, we generate high-quality queries without human annotations, significantly enhancing retrieval performance in conversational recommendation systems.

3.3 The Improved Recommendation Stage Rationalize-Then-Recommend (RTR)

The primary objective of the recommendation stage is to capture subtle user preference information from the original conversation that may be difficult for the retriever to detect [10,17,18]. By combining the item descriptions from the small candidate pool provided by the retrieval stage, this phase outputs the final recommendation to the user.

While many existing LLM-based CRS models [6,23] either overlook sentence generation or handle the recommendation and sentence generation tasks separately, we aim to seamlessly integrate them. To achieve this, we propose a fine-tuning task called **Rationalize-then-Recommend (RTR)**. The RTR task trains the LLM to: (1) effectively extract user preferences from noisy, historical conversations to make recommendations, (2) identify connections between user preferences and the given item descriptions, and (3) provide natural language responses that justify its recommendations (Fig. 4).

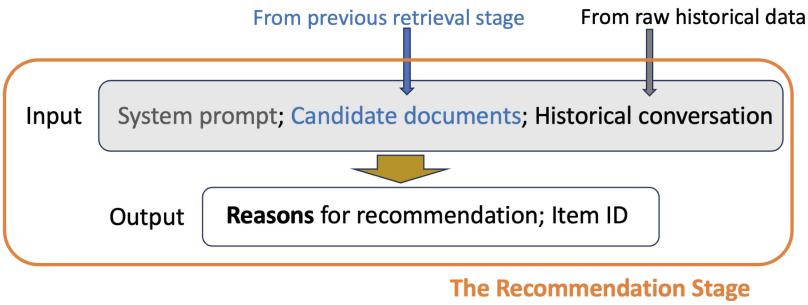

Fig. 4. The "Rationalize-then-Recommend" (RTR) fine-tuning task designed for accomplishing conversational recommendation efficiently and effectively.

To mitigate hallucinations, we replace item names (e.g., movie titles) with IDs ranging from 1 to K, where K is the number of candidate items. The LLM is

trained to recommend the correct item by selecting its corresponding ID, rather than generating item names.

During recommendation, the input to the RTR-trained LLM consists of: (1) a system prompt summarizing the task, (2) candidate item descriptions $\mathcal{D}_i^{\text{top}} = \{\mathbf{d}_1, \mathbf{d}_2, \ldots, \mathbf{d}_K\}$ retrieved earlier with item names replaced by IDs, including hard negatives to differentiate true preferences from misleading ones, and (3) the conversation history C_i representing user preferences.

The LLM generates a response with two components: a rationale explaining the identified preferences and relevant concepts from the conversation, and a recommendation specifying the chosen item ID. Since the LLM outputs token sequences, it is trained using maximum likelihood estimation (MLE) to maximize the probability of generating the correct rationale and recommendation, equivalent to minimizing the cross-entropy loss over the sequence.

Given an input, the LLM produces the response $y_i = \{y_i^{(1)}, y_i^{(2)}, \ldots, y_i^{(T_i)}\}$, where T_i is the response length. We minimize the negative log-likelihood:

$$\mathcal{L}_{\text{RTR}} = -\sum_i \sum_{t=1}^{T_i} \log P_{\text{LLM}_{\text{RTR}}}(y_i^{(t)} \mid y_i^{(<t)}, \text{Input}_i) \tag{1}$$

Here, $y_i^{(<t)}$ denotes all preceding tokens, and Input_i includes the system prompt, candidate documents, and conversation history for sample i. During inference, the LLM predicts tokens sequentially to produce both rationale and recommendation:

$$y_i = \text{LLM}_{\text{RTR}}(\text{Input}_i) \tag{2}$$

Training the RTR model independently focuses on enhancing its ability to generate coherent, justified recommendations based on input data.

Avoiding Hallucinations. To further mitigate hallucinations, we enforce constraints during generation. For instance, we can use a constrained decoding strategy where the LLM is only allowed to generate item IDs within the valid range $\{1, 2, \ldots, K\}$. Additionally, we can include a verification step that checks whether the generated recommendation corresponds to one of the candidate items.

4 Experiments

In this section, we first report the dataset we used. Then, we describe the experiments for retrieval stage highlights the effectiveness of CTS supervision, followed by reporting the results of the recommendation and providing a detailed analysis of RTR fine-tuning task.[1]

Dataset: We conduct experiments on two widely used CRS datasets: ReDial [11] and INSPIRED [5]. For fair comparison, we directly borrow the raw data preprocessing pipeline for ReDial and INSPIRED dataset from previous studies [25].

[1] Code will be publicly available after publication.

4.1 The Retrieval Stage CTS

This section presents the retrieval stage results. Strong retrieval performance is crucial for accurate recommendation generation, as the latter depends on the former in RAG.

Experiment Setup Evaluation Metrics: The retrieval stage aims to provide fresh knowledge and filter out sub-relevant items, ensuring the genuine recommendation is included in the candidate pool. If the correct recommendation is missing, the recommendation stage fails. Balancing efficiency and effectiveness, we set candidate pool sizes to 10 and 50, and use Recall@10 and Recall@50 to assess retrieval performance.

Implementation Details: We use GPT-3.5-turbo for the "Compare then Summarize" (CTS) pipeline to generate pseudo query reformulations and as the base model for building the query reformulator. We collected 200 human-annotated reformulations from INSPIRED and ReDial datasets, where annotators converted historical conversations into concise queries representing user preferences. For consistency, CTS-generated reformulations were created only for these 200 samples to enable a fair comparison.

We explore two supervision strategies: (i) fine-tuning GPT-3.5 on the 200 samples using OpenAI's API (learning rate multiplier = 2, epochs = 2, batch size = 4), and (ii) few-shot prompting using a subset of the 200 samples. To evaluate model robustness across varying capabilities, we use three frozen embedding models: DPR [10], BGE-large-v1.5-en [17], and OpenAI's Text-embedding-3-large, with BGE-large-v1.5-en as our primary model.

Experiments involve comparing different query reformulation strategies: using the raw user-system conversation as the query (**Originconv**), zero-shot prompting GPT-3.5 (**ZS-GPT3.5**), and fine-tuning GPT-3.5 on all human-annotated (**Human + FT GPT3.5**) or CTS-generated (**CTS + FT GPT3.5**) samples. We also test few-shot prompting with 5 human (**Human + 5S GPT3.5**) or CTS (**CTS + 5S GPT3.5**) examples. In all cases, the same prompt is used: "*Generate a search query for matching a movie that the SEEKER may like*".

Experiment Results. Table 1 presents the performance of the retrieval stage using CTS-generated supervision compared to human supervision. Our first observation is that using the original conversation as a query consistently results in poor retrieval performance, regardless of the metric or dataset used, underscoring the importance of query reformulation.

When comparing query reformulators built with human versus CTS-generated supervision, we find that CTS supervision outperforms human supervision on the INSPIRED dataset, even though CTS is significantly cheaper to generate. On the ReDial dataset, where less user preference information is available, CTS still delivers query reformulation quality comparable to human annotations.

Table 1. Retrieval performance comparison.

Experiment name	INSPIRED		ReDial	
	Recall@10	Recall@50	Recall@10	Recall@50
Originconv	0.1491	0.3114	0.1034	0.2148
ZS-GPT3.5	0.1974	0.4254	0.1471	0.2887
Human + FT GPT3.5	0.1623	0.3772	0.1223	0.2976
Human + 5S GPT3.5	0.2105	0.4254	**0.1768**	**0.3371**
CTS + FT GPT3.5 (Ours)	0.2237	0.4211	0.1143	0.2754
CTS + 5S GPT3.5 (Ours)	**0.2719**	**0.4956**	0.1652	0.3067

Interestingly, in terms of supervision methods, few-shot prompting surprisingly outperforms full parameter fine-tuning.

Table 2. Retrieval performance across various embedding models.

INSPIRED							
Embedding model	Embedding Dim	Experiment name	hit@10	hit@50	Experiment name	hit@10	hit@50
DPR	512	Originconv	0.0570	0.2018	CTS+ 5S GPT3.5	0.1842	0.3816
BGE	1024		0.1491	0.3114		0.2719	0.4956
Text-embedding-3-large	4096		0.2061	0.3596		0.3070	0.4912
ReDial							
Embedding model	Embedding Dim	Experiment name	hit@10	hit@50	Experiment name	hit@10	hit@50
DPR	512	Originconv	0.0459	0.1134	CTS+ 5S GPT3.5	0.1014	0.2156
BGE	1024		0.1034	0.2148		0.1652	0.3067
Text-embedding-3-large	4096		0.1616	0.3310		0.1809	0.3479

We further evaluated whether CTS-generated supervision consistently enhances performance across embedding models of varying capacities. In addition to BGE, we assessed retrieval performance on a more efficient but less powerful model, DPR [10], and a more advanced model, OpenAI's Text-embedding-3-large [13]. Table 2 shows that models with larger embedding dimensions generally outperform smaller ones. The results also emphasize the importance of query reformulation, especially for lightweight models like DPR. Notably, using a CTS-supervised query reformulator significantly improves retrieval over the original conversation as a query. Moreover, CTS-generated supervision demonstrates generalizability and robustness across models of varying sizes.

Discussion: Different Approaches of Providing Supervision. Figure 5 presents the retrieval performance (Recall@10 and Recall@50) across various supervision methods for the LLM-based query reformulator. The results highlight the effectiveness of few-shot prompting and the value of long samples. Few-shot prompting generally outperforms full-parameter fine-tuning, likely due to

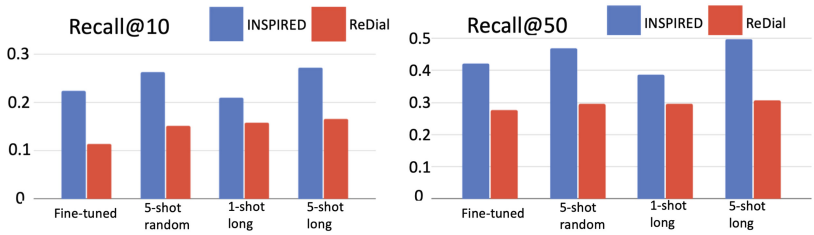

Fig. 5. Comparing the retrieval performance (Recall@10 & Recall@50) under different ways of providing supervision to LLMs. Random = examples are randomly selected; Long = examples are those with longest length.

reduced overfitting risks given the limited size of the supervision data. Furthermore, incorporating long samples consistently enhances performance, as they better capture user preferences, helping the query reformulator grasp nuanced information during inference.

4.2 Overall Performance

In this section, we show the overall performance of our proposed RAGCRS. Since experiment details of the retrieval stage is reported in the last section, we focus more on the experimental details of the recommendation stage.

Experiment Setup. We use Llama3-8B-instruct [1] as the base model for the recommendation stage, balancing computational efficiency and performance under resource constraints. The model is fine-tuned on the "Rationalize-then-Recommend" (RTR) task, with reasoning parts pre-generated by GPT-3.5-turbo. Fine-tuning employs a batch size of 1 and a maximum of 2 epochs, as evaluation performance converged quickly. To reduce GPU memory usage, we apply LoRA (Low-Rank Adaptation) with a rank of 8 and alpha of 32, modifying only the query and value matrices in the attention mechanism. For baselines, we implemented KGSF and KBRD via CRSLab and used the official UniCRS repository. Since LLM-CRS [6] did not release their code, we directly use the reported results without adjusting for hallucinated items to ensure fair comparison.

Table 3. Overall comparison of Recommendation Success Rate.

Method		Rec Success Rate	
		INSPIRED	ReDial
Non-LLM-based	KBRD	0.0357	0.0268
	KGSF	0.0510	0.0300
LLM-based	LLM-CRS	0.0470	0.0410
	RAGCRS	**0.0859**	**0.0422**

Evaluation Metrics: The primary evaluation metric is "success recommendation rate", equivalent to recall@1 or Hit@1 used in prior work.

Baselines: We compare against four recent CRS methods: **KBRD** [2], leveraging the DBpedia knowledge graph for domain knowledge; **KGSF** [26], combining DBpedia with ConceptNet for both domain-specific and common-sense knowledge; and **LLM-CRS** [6], prompting GPT-3.5 directly with the full conversation history for movie recommendations.

Experiment Results. Table 3 shows the overall performance comparison between our proposed RAGCRS and baseline methods in terms of recommendation success rate. In the INSPIRED dataset, where the conversations are more complex, involve noisier social interactions, and contain more user preference exposure, our proposed method achieves a performance increase of up to 80% compared to the existing state-of-the-art LLM-based method, LLM-CRS.

Table 4. Comparing Average Hallucinate Ratio.

	Model Size	Rec. Gen. Style	Avg Hallu. Ratio
RAGCRS	8B	ID	0%
		Item Name	0.13%
LLM-CRS	∼20B (GPT3.5)	Item Name	4.49%
	∼200B (GPT4)	Item Name	5.14%

A key challenge limiting the industrial adoption of LLM-based CRS is hallucinate. To address this, we improved the RTR fine-tuning task by replacing item names with numeric IDs. Each ID corresponds to a candidate from the retrieval stage, indexed from 1 to the size of the candidate pool (e.g., for a pool of 10 candidates, IDs range from 1 to 10). As shown in Table 4, while retrieval-augmented generation significantly lowers the hallucination rate from 5% to 0.13%, switching from item names to IDs reduces it to zero across all scenarios.

4.3 Discussion: Size of Candidate Pool

Table 5 demonstrates the effectiveness of the RTR fine-tuning task in improving recommendation success across various candidate pool sizes. While larger candidate pools from the retrieval stage increase the chance of containing the ground truth item (Recall), they also raise the difficulty for the LLM to select the correct recommendation and escalate inference costs. Notably, the RTR fine-tuned LLM maintains strong performance in selecting the correct recommendation, even with increasing pool sizes.

Table 5. Conversational recommendation success rate providing different size of candidate pool to RTR-fine-tuned LLM

#Candidates	INSPIRED	ReDial
	Rec Success Rate	
50	0.0859	0.0422
10	0.0647	0.0291

By default, RTR fine-tuning trains the LLM to generate both the reasoning behind a recommendation and the recommendation itself (i.e., item name), integrating these subtasks to enhance the efficiency of CRS. We also explored generating only the recommendation without reasons, which improved accuracy on both the INSPIRED and ReDial datasets. This aligns with prior findings [16] that including reasoning may hinder smaller models' performance but benefits larger models (above 8B parameters).

5 Conclusion

In this study, we introduced RAGCRS that enables the use of RAG to address three major limitations in existing LLM-based CRS methods. Our approach enhances system effectiveness by boosting recommendation success rates. The "Compare-then-Summarize" (CTS) and "Rationalize-then-Recommend" (RTR) techniques are crucial in RAGCRS, ensuring accurate retrieval and successful recommendations, respectively.

Looking ahead, we plan to incorporate behavior modeling to refine the recommendation module's behavior. Additionally, we aim to broaden the retrieval scope from text to other data forms, paving the way for a multi-modal RAGCRS.

References

1. Meta AI. Introducing meta LLaMA 3: the most capable openly available LLM to date (2024). https://ai.meta.com/blog/meta-llama-3/
2. Chen, Q., et al.: Towards knowledge-based recommender dialog system. arXiv preprint arXiv:1908.05391 (2019)
3. Di Palma, D.: Retrieval-augmented recommender system: enhancing recommender systems with large language models. In: Proceedings of the 17th ACM Conference on Recommender Systems, pp. 1369–1373 (2023)
4. Gao, C., et al.: Advances and challenges in conversational recommender systems: a survey. AI Open **2**, 100–126 (2021)
5. Hayati, S.A., et al.: Inspired: toward sociable recommendation dialog systems. arXiv preprint arXiv:2009.14306 (2020)
6. He, Z., et al.: Large language models as zero-shot conversational recommenders. In: Proceedings of the 32nd ACM International Conference on Information and Knowledge Management, pp. 720–730 (2023)

7. Hua, W., et al.: Tutorial on large language models for recommendation. In: Proceedings of the 17th ACM Conference on Recommender Systems, pp. 1281–1283 (2023)
8. Jannach, D., et al.: A survey on conversational recommender systems. ACM Comput. Surv. (CSUR) **54**(5), 1–36 (2021)
9. Ji, Z., et al.: Survey of hallucination in natural language generation. ACM Comput. Surv. **55**(12), 1–38 (2023)
10. Karpukhin, V., et al.: Dense passage retrieval for open-domain question answering. arXiv preprint arXiv:2004.04906 (2020)
11. Li, R., et al.: Towards deep conversational recommendations. In: Advances in Neural Information Processing Systems, vol. 31 (2018)
12. Lin, J., Nogueira, R., Yates, A.: Pretrained Transformers for Text Ranking: BERT and Beyond. Springer (2022)
13. OpenAI. New Embedding Models and API Updates (2024). https://openai.com/index/new-embedding-models-andapi-updates/. Accessed 06 Sept 2024
14. Team, G., et al.: Gemma: open models based on Gemini research and technology. arXiv preprint arXiv:2403.08295 (2024)
15. Touvron, H., et al.: LLaMA 2: open foundation and fine-tuned chat models. arXiv preprint arXiv:2307.09288 (2023)
16. Wei, J., et al.: Chain-of-thought prompting elicits reasoning in large language models. In: Advances in Neural Information Processing Systems, vol. 35, pp. 24824–24837 (2022)
17. Xiao, S., et al.: C-pack: packaged resources to advance general Chinese embedding. arXiv: 2309.07597 [cs.CL] (2023)
18. Xiong, L., et al.: Approximate nearest neighbor negative contrastive learning for dense text retrieval. arXiv preprint arXiv:2007.00808 (2020)
19. Yang, D., Chen, F., Fang, H.: Behavior alignment: a new perspective of evaluating LLM-based conversational recommendation systems. In: Proceedings of the 47th International ACM SIGIR Conference on Research and Development in Information Retrieval, SIGIR 2024, pp. 2286–2290. Association for Computing Machinery, Washington DC (2024). https://doi.org/10.1145/3626772.3657924. ISBN 9798400704314
20. Yang, D., Zhang, Y., Fang, H.: An exploration study of mixedinitiative query reformulation in conversational passage retrieval. arXiv preprint arXiv:2307.08803 (2023)
21. Yang, D., Zhang, Y., Fang, H.: Zero-shot query reformulation for conversational search. In: Proceedings of the 2023 ACM SIGIR International Conference on Theory of Information Retrieval, ICTIR 2023, pp. 257–263. Association for Computing Machinery, Taipei (2023). https://doi.org/10.1145/3578337.3605143. ISBN: 9798400700736
22. Yates, A., Nogueira, R., Lin, J.: Pretrained transformers for text ranking: BERT and beyond. In: Proceedings of the 14th ACM International Conference on Web Search and Data Mining, pp. 1154– 1156 (2021)
23. Zhang, G.: User-centric conversational recommendation: adapting the need of user with large language models. In: Proceedings of the 17th ACM Conference on Recommender Systems, pp. 1349–1354 (2023)
24. Zhang, J., et al.: Is chatGPT fair for recommendation? Evaluating fairness in large language model recommendation. In: Proceedings of the 17th ACM Conference on Recommender Systems, pp. 993–999 (2023)
25. Zhou, K., et al.: CRSLab: an open-source toolkit for building conversational recommender system. arXiv preprint arXiv:2101.00939 (2021)

26. Zhou, K., et al.: Improving conversational recommender systems via knowledge graph based semantic fusion. In: Proceedings of the 26th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining, pp. 1006– 1014 (2020)
27. Zhu, Y., et al.: Collaborative large language model for recommender systems. In: Proceedings of the ACM on Web Conference 2024, pp. 3162–3172 (2024)